BPEトークナイザを自作する

参考資料 — ChatGPT の語彙はこうやって作られている

rkskmt

なぜ文字をやめたくなるか

- 本編の勘定（深く積む／ChatGPTまでの距離）を思い出す：
  → 文字レベル日本語は語彙3,045 — **パラメータの7割**が語彙の出入口に取られた
  → 窓（block_size=64）に**64文字**しか入らない — 長い話が読めない
- 語彙を「文字」より大きい単位にできれば、両方が一度に楽になる
- 実物の LLM が使う単位：**サブワード**。その標準的な作り方が **BPE**（**Byte Pair Encoding**）

アイデアは1行

> ⓘ **BPE のすべて**
>
> **一番よく隣り合うペアを、合体して新しい1トークンにする。** これを繰り返すだけ。

- 「数える」のは隣り合う2トークンの回数 — **バイグラム表の数え上げと同じ技**
- 合体するたびに語彙が1つ増え、文章は少し短くなる
- 好きな語彙サイズになったらやめる（GPT-2 は約5万で停止）

実装 — 20回合体させる

Code

```
from collections import Counter

text = open("data/neko.txt", encoding="utf-8").read()[:100000]   # 前半10万字で実験
tokens = list(text)                          # 最初は1文字=1トークン

merges = []
for step in range(20):
    pairs = Counter(zip(tokens, tokens[1:]))     # 隣接ペアを数える
    (a, b), n = pairs.most_common(1)[0]          # 最多ペア
    merges.append(a + b)
    new_tokens, i = [], 0
    while i < len(tokens):                       # 合体して列を作り直す
        if i < len(tokens) - 1 and tokens[i] == a and tokens[i + 1] == b:
            new_tokens.append(a + b)
            i += 2
        else:
            new_tokens.append(tokens[i])
            i += 1
    tokens = new_tokens

print("覚えた合体:", merges)
print("トークン数: 100000 →", len(tokens))
```

Result

```
覚えた合体: ['って', 'る。', 'して', 'ない', 'から', 'った', '」と', 'した', '」「', 'ると', '主
人', 'です', 'よう', 'であ', 'もの', '云う', 'でも', 'ます', 'その', '吾輩']
トークン数: 100000 → 91366
```


機械が発見した「単語」

- 20回の合体で生まれたトークンを眺める：

種類	生まれたトークン
活用語尾・助詞のかたまり	って／した／ない／から／です／ます
会話文の型	」と／」「（かぎ括弧の作法まで学んだ）
単語そのもの	**主人／吾輩／云う**

- 単語の区切りを誰も教えていないのに、**頻度だけで「吾輩」が1トークンになった**
- ことばの地図の回と同じ結論 — 統計は構造を勝手に見つける

> ⓘ **ノート**
>
> 実物の BPE は文字ではなく**バイト列**から始める（byte-level BPE）。 絵文字も外国語も取りこぼさないための工夫で、原理はこのページと同じ。

語彙とのトレードオフ

- 合体を続けるほど：文章は短くなる（窓に長い話が入る）／語彙は増える（出入口が重くなる）

合体回数	語彙サイズ	10万字のトークン数
0（文字のまま）	約2,300	100,000
20	+20	91,366
数万（GPT-2 相当）	50,257	（日本語なら数万に縮む）

- 「どこでやめるか」は設計判断 — 実物は数万で手を打っている

> **ヒント**
>
> LLM の料金や文脈長が「トークン数」で数えられている理由が、これで足元から説明できる。

演習：合体を伸ばす

本編のコード の合体回数を増やして、語彙の育ち方を観察せよ。

0. **neko.txt** を `data/` に配置し、本編のコードを実行
1. `range(20)` を `range(100)` にして再実行（数秒〜数十秒）
2. `merges` の中で**一番長いトークン**を表示（`max(merges, key=len)`）
3. トークン数の減り方（100,000 → ?）も表示

▸ 解答例を見る